

Experiment Name

Write a program to compute FIRST of terminals and non-terminals of a given Context-Free Grammar (CFG).

Objective

To compute the FIRST set for all terminals and non-terminals in a given context-free grammar (CFG). The FIRST set of a symbol X is the set of terminals that appear as the first symbols in strings derived from X .

Algorithm

1. Start the program.
2. Read the number of production rules from the user. For each production, read the left-hand side (non-terminal) and the right-hand side (sequence of symbols), and store them in the grammar.
3. Identify all terminals and non-terminals from the grammar:
 - Any symbol appearing on the left-hand side of a production is a **non-terminal**.
 - Any symbol that is not a non-terminal is a **terminal** (including ϵ).
4. Initialize FIRST sets:
 - For each terminal a , set $\text{FIRST}(a) = \{a\}$, since a terminal derives only itself.
 - For each non-terminal A , initialize $\text{FIRST}(A) = \emptyset$ (empty set).
5. Apply the following fixed-point iteration — repeat until no FIRST set changes:
 - For each production rule $A \rightarrow X_1 X_2 X_3 \cdots X_n$, process each symbol X_i from left to right:
 - **Case 1:** If X_1 is a terminal a (or ϵ), add a (or ϵ) directly to $\text{FIRST}(A)$ and stop processing this production.
 - **Case 2:** If X_i is a non-terminal B , add all symbols in $\text{FIRST}(B) - \{\epsilon\}$ to $\text{FIRST}(A)$.
 - **Case 3:** If $\epsilon \in \text{FIRST}(X_i)$, then X_i is nullable — continue to the next symbol X_{i+1} and repeat.
 - **Case 4:** If $\epsilon \notin \text{FIRST}(X_i)$, stop processing this production — the remaining symbols cannot contribute to $\text{FIRST}(A)$.

- **Case 5:** If all symbols $X_1, X_2, \dots, X_n$ are nullable (i.e., ε is in all their FIRST sets), then add ε to $\text{FIRST}(A)$.
6. After the iteration converges (no more changes occur), the FIRST sets are complete.
 7. Display the computed FIRST sets for all non-terminals.
 8. End the program.

Input Grammar

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \varepsilon$$

$$F \rightarrow (E) \mid \mathbf{id}$$

Source Code

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  map<string, vector<vector<string>>> G;
5  map<string, set<string>> FIRST;
6
7  bool NT(string s){ return isupper(s[0]); }
8
9  vector<string> split(string s){
10     vector<string> v; string w; stringstream ss(s);
11     while(ss>>w) v.push_back(w);
12     return v;
13 }
14 int main(){
15     int n; string line,prod;
16     cout<<"Enter number of productions: ";
17     cin>>n; cin.ignore();
18
19     cout<<"Enter productions:\n";
20     while(n--){
21         getline(cin,line);
22         int p=line.find("->");
23         string L=line.substr(0,p-1), R=line.substr(p+3);
24         stringstream ss(R);
25         while(getline(ss,prod,'|')) G[L].push_back(split(prod));
26     }
27
28     for(auto &g:G) FIRSTof(g.first);
29
30     cout<<"\nFIRST sets:\n";
31     for(auto &g:G){
32         cout<<"FIRST("<<g.first<<" ) = { ";
33         for(auto &x:FIRST[g.first]) cout<<x<<" ";
34         cout<<"}\n";
35     }
36 }
```

Figure 1: Source code for computing FIRST sets of a given grammar

Output Screenshot



```
PS C:\Users\Admin\Documents\1093> cd "c:\Users\Admin\Documents\1093\" ; if ($?) { g++ ccl7.cp
p -o ccl7 } ; if ($?) { .\ccl7 }
Enter number of productions: 5
Enter productions (e.g., E -> T E'):
E -> T E'
E' -> + T E' | ε
T -> F T'
T' -> * F T' | ε
F -> ( E ) | id

FIRST sets:

FIRST(E) = { ( id }
FIRST(E') = { + e }
FIRST(F) = { ( id }
FIRST(T) = { ( id }
FIRST(T') = { * e }
PS C:\Users\Admin\Documents\1093> 
```

Figure 2: Output showing computed FIRST sets

Discussion

This experiment demonstrates the implementation of the FIRST set computation algorithm for context-free grammars. The FIRST set is a fundamental concept in compiler construction, particularly used in the design of top-down parsers such as the LL(1) parser.

The program correctly handles all cases: productions starting with a terminal (directly added to the FIRST set), productions starting with a non-terminal (FIRST of that symbol minus ϵ is propagated), nullable non-terminals (algorithm looks ahead to the next symbol), and epsilon productions (added when all symbols in a production are nullable).